Chart library

User manual

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

Table of Contents

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

1 Introduction

This library allows you to visualize data by filling the chart object with data. This library can be filled with data, has custom titles, has customizeable descriptions for the y-axis, allows color styling, has different chart types such as line, bar, column, pie and many more. This library functions as a wrapper for some of the more well-known charting libraries such as Chartjs, Highcharts and Syncfusion.

Each of the charting libraries support a different set of available chart types, the available types are listed below.

Chartjs

- Line
- Bar
- Doughnut
- Pie
- PolarArea
- Radar
- Scatter
- Bubble

Highcharts

- Line
- Bar
- Column
- Area
- Pie
- Doughnut
- Areaspline
- Scatter
- Spline

Syncfusion

- Line
- Bar
- Column
- Radar
- StepLine
- Spline
- Area
- StackingArea
- StackingArea100
- StackingStepArea
- StepArea
- SplineArea
- StackingColumn
- StackingColumn100
- StackingBar
- StackingBar100
- Scatter
- Bubble

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

- Polar
- Radar
- Pareto

Attention!
The use of the open source Chartjs library is free to use. However, if you want to implement a Highcharts or Syncfusion chart in your application you will need to get a license for the specific library on their website based on your needs. DataAccess only offers a way to easily use these libraries and does **NOT** provide any licensing.

1.1 Adding the library to your workspace

The library is installed with JavaScript files. To use these JavaScript files in your workspace you will need to move the Charts folder to the AppHtml folder located in your workspace. After this they will need to be included in Index.html. Depending on what charting libraries you include you will need to include different files. The source files for highcharts and syncfusion are not automatically downloaded because of licensing. You can find the downloads for those files in the following places:

- https://www.highcharts.com/download/
- https://www.syncfusion.com/javascript-ui-controls/js-charts

Once you have downloaded these files, make sure to add them to your apphtml folder and replace the “Path to xxxx js file here” below to a path of the js file you downloaded. It is important that the highcharts/syncfusion source files are included **BEFORE** the renderer.js class.

One file that should always be included is the ChartWrapper.js file. This can be done like so:

```
<script type="module" src="Charts/ChartControl.js"></script>
```

In case you want to use chartjs in your application include the following lines:

```
<script src="Charts/Chartjs/chart.min.js"></script>

<script type="module" src="ChartWrappers/Chartjs/ChartjsRenderer.js"></script>
```

In case you want to include Highcharts in your application include the following lines:

```
<script src="Path to highcharts js file here "></script>

<script type="module" src="Charts/HighCharts/HighChartsRenderer.js"></script>
```

In case you want to use Syncfusion in your application include the following lines:

```
<script src="Path to syncfusion js file here"></script>

<script type="module" src="Charts/Syncfusion/SyncfusionRenderer.js"></script>
```

It is possible to include multiple different libraries at once, the control allows you to switch between them by setting psChartingLibrary to the library you want to use. Now that the Javascript files have been added to the workspace all that is left to do is to add the DataFlex library to your workspace. To do this go into your studio, select tools, then click maintain libraries. After you have clicked maintain libraries a screen will pop-up. On this screen select “Add Library”. Now search for the location where the library is installed on your computer and select the .sws file that is compatible with your DataFlex version.

1.2 How does it work?

The chart library makes it possible to visualize data by filling a struct with data and then calling the CreateChart procedure along with the filled struct. To start making charts you will need to start by creating a tChartData struct like so:

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

tChartData[] chartData

The tChartData struct has a few mandatory properties that need to be filled and a few optional properties that need to be filled. Let's start with the mandatory properties.

Mandatory

sLabel

You can set the series name by setting sLabel of the chartData like so:

Move "Sample data" to chartData[0].sLabel

This defines the name of that specific dataset. This name is displayed in the legend. The name is also displayed when hovering over your dataset in a chart.

nData

nData is a list of all the data in your chart nData is an array of numbers. So, to fill the array you would fill it like you would fill a regular DataFlex array. By specifying a specific index for the data. Or by appending it to the end of the array by using -1 as the index.

Optional

sSeriesColor

You can set the color of your data by defining sSeriesColor. This can be done in the following ways:

Move "red" to chartData[0].sSeriesColor

Move "rgba(255, 255, 255, 0.5)" to chartData[0].sSeriesColor

Move "rgb(255, 255, 255) to chartData[0].sSeriesColor

If sSeriesColor is not specified, a default color will be selected for the data. The color selected differs for each different charting library

sType

SType determines the type of chart your data will be shown as. For example, setting sType to line will make sure that the data will be displayed as a line chart. This allows a single chart object to have both line and bar charts for example.

1.3 Licensing

Data Access Worldwide provides wrappers for developers to use these libraries in their applications. However, Data Access Worldwide does not provide the developers with the licenses to use these libraries. If you want to implement any of these libraries look at the licensing options listed below.

1.3.1 Chartjs

Chartjs is made available under the MIT license and is considered as open-source software. This library can be used in your application if you provide the MIT license within your application.

1.3.2 Highcharts

Highcharts is available under a paid license and is not free for commercial purposes. If you want to use a Highcharts chart within your application, you should look at the licenses available at https://shop.highcharts.com/?gad_source=1&gclid=CjwKCAiA-vOsBhAAEiwAIWR0Tcvg3HTh2hawyRU0MndaE4iIHuHJJftGEgEt7yeQwlu4EfdGAjHeBBoCmz0QAvD_BwE.

1.3.3 Syncfusion

Syncfusion is available under a paid license and is not free for commercial purposes. However, if your company is eligible for the community license it can be used for free as long as you stay eligible for the community license. To see more about the community license see https://www.syncfusion.com/products/communitylicense. If your company does not fall within the community license and you would like to implement a Syncfusion chart in your application, you will need to purchase a license here https://www.syncfusion.com/sales/teamlicense.

2 Creating a chart

When the chart is initialized, it will be empty because no data has been passed to the chart object yet. To create a chart for the first time the CreateChart procedure needs to be called. The CreateChart procedure takes a tChartData array as parameter. Each item in the array represents an individual data series. So, for example if you call the CreateChart procedure with 2 items inside of the tChartData array, 2 lines will be drawn in a line chart.

3 Interacting with the chart

There are multiple ways to interact with the chart. Here is a list of the available interactions:

- Add new series
- Add new datapoints
- Change existing datapoints
- Remove datasets
- Get information from the data you click on

3.1 Adding new series

Adding new series to the chart is done with the AddNewChartSeries procedure. This procedure takes a tChartData as argument. This tChartData represents the new dataseries that has to be added to the chart.

3.2 Adding new datapoints

Adding new datapoints to an existing data series is done through the AddNewDataPoint procedure. This procedure takes 3 arguments. The first argument is the name of the dataset that the point should be added to, the second argument is where on the x axis the point should be added to. If this argument does not exist on the x axis it will be added at the end of the dataset and a new value will be added to the x axis. The last argument is the actual value of the datapoint to be added. This value is a number value.

3.3 Change existing datapoint

Changing an existing datapoint is done through the ChangeDatapoint procedure. This procedure takes 3 arguments. The first argument is the name of the dataset that the datapoint belongs to. The second argument is the index of the value that is going to be changed. So, if the first number of a dataset should be changed the index would be 0 like a regular array. The last argument is the value that the datapoint should be changed to. The value is a number value.

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

3.4 Remove datasets

Removing datasets from the chart is done through the RemoveDataset procedure. This procedure takes a single argument. This argument is the name of the dataset that should be removed from the chart. This value is in string format. After calling this procedure the dataset will be removed from the chart.

3.5 On click information

Getting information from the chart happens through the OnClick event. The OnClick event is called whenever a datapoint is clicked. The OnClick event sends the following information.

- The value of the x axis as a string
- The value as a number
- The index of the value in its dataset as an integer
- The name of the dataset as a string

4 Properties for cChart

Name	Description	Default value
cChart		
psTitle	The title displayed above the chart in bold.	
psSubtitle	The subtitle displayed above the chart, this is shown under the title and is not bold.	
psChartType	The type of the chart that should be displayed. This value is used when the type is not set for a dataset during creation.	line
psChartBackgroundColor	Sets the background color of the chart	
psYAxisLabel	Sets a description that is displayed on the y axis.	
psXAxisLabels	Sets the values of the x axis label. Values should be separated by commas.	
psChartingLibrary	The charting library used. Current available options are Chartjs, Highcharts and Syncfusion.	Chartjs
psLegendAlignment	Determines the position of the legend. Available options are left, top and right.	right
pbLegendEnabled	Determines if the legend is shown to the user.	True
pbZoomable	Determines if the user can zoom and pain in the chart.	True

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu

5 Methods for cChart

Title: UpdateChart	
Type: Procedure	
Description	Fully redraws the existing chart.

Title: CreateChart		
Type: Procedure		
Description	Creates a chart	
Parameters		
Name	**Description**	**Type**
chartData	The information of all the datasets that are included in the chart.	tChartData[]

Title: AddNewChartSeries		
Type: Procedure		
Description	Adds a new dataset to the chart	
Parameters		
Name	**Description**	**Type**
chartData	The information of the dataset that has to be added to the chart.	tChartData

Title: AddNewDataPoint		
Type: Procedure		
Description	Adds a new datapoint to an existing dataseries	
Parameters		
Name	**Description**	**Type**
sDatasetname	The name of the dataset the datapoint should be added to,	String
sXAxisLabel	The name of the x axis value where the datapoint will be inserted. If this value doesn't already exist on the chart the datapoint will be added to the end.	String
Data	The value of the datapoint.	tPointData

Title: ChangeDatapoint		
Type: Procedure		
Description	Changes an existing datapoint	
Parameters		
Name	**Description**	**Type**
sDatasetName	The name of the dataset the datapoint belongs to.	String
iIndex	The index of the value in the dataset.	Integer
nNewValue	The new value of the datapoint.	Number

Title: RemoveDataset		
Type: Procedure		
Description	Remove a dataset from the chart	
Parameters		
Name	Description	Type
sDatasetName	The name of the dataset.	String

6 Events for cChart

Title: OnClick		
Type: Event procedure		
Description	Receive a mouse click event	
Parameters		
Name	Description	Type
sLabel	The value of the x axis that the datapoint is on.	String
nValue	The value of the datapoint that was clicked.	Number
iIndex	The index the datapoint has in the dataset that it's in.	Integer
sDatasetName	The name of the dataset that the datapoint is in.	String

7 Structs for cChart

Title: tChartData		
Type: Struct		
Description	The data represents a dataseries in a chart.	
Members		
Name	Description	Type
sLabel	The name of the dataset.	String
sSeriesColor	The color of the dataset.	String
dataPoints	The data inside of the dataset.	tPointData[]
sType	The type of chart the dataset is.	String
nLineThickness	The thickness of the line in a line chart.	Number

Title: tPointData		
Type: Struct		
Description	This struct represents a data point.	
Members		
Name	Description	Type
Y	The actual value of the point.	Number
sTooltip	The tooltip that is shown when hovering over the point.	String

Data Access Europe BV / Lansinkesweg 4 / 7553 AE Hengelo, The Netherlands +31 (0)74 255 56 09 info@dataaccess.eu www.dataaccess.eu